Reverse Substrings Between Each Pair of Parentheses

Company: Microsoft
Round: Online Assessment (OA)
Year: 2025
Difficulty: Medium
Topic: Strings, Stack, Two-Pointer / Direction-Jump Technique
Source: LeetCode 1190

Problem Description

Given a string s that consists of lowercase English letters and parentheses, reverse the substring inside every pair of matching parentheses, starting from the innermost pair outward. The final output string must not contain any parentheses.

Input Format

- A single line containing the string `s`, made up of lowercase letters `a–z`, `(`, and `)`. The parentheses are guaranteed to be balanced.

Output Format

- Print the resulting string after all reversals, with all parentheses removed.

Constraints

- $1 \leq s.length \leq 2000$
- s consists of lowercase English letters and the characters (and)
- Every opening bracket has a matching closing bracket, and every closing bracket has a matching opening bracket (the brackets always form a valid sequence).

Sample Input / Output

Example 1

Input:

(skeeg(for)skeeg)

Output:

geeksforgeeks

Example 2

Input:

((ng)ipm(ca))

Output:

camping

Example 3

Input:

ab(cd)

Output:

abdc

Explanation

In Example 1, "(skeeg(for)skeeg)" → "geeksforgeeks"
The innermost pair (for) is reversed first to give rof, turning the string into (skeegrofskeeg). Reversing the outer pair then flips the whole enclosed segment skeegrofskeeg, which lands us at geeksforgeeks. Working from the innermost bracket outward guarantees each character ends up facing the correct direction by the time all brackets are resolved.

In Example 2, "((ng)ipm(ca))" → "camping"
There are two innermost pairs here: (ng) becomes gn, and (ca) becomes ac. After resolving both, the string becomes (gnipmac). Reversing this outer pair flips the entire segment gnipmac, giving camping. Since the two inner pairs are independent of each other, they can be resolved in either order before the outer reversal is applied.

In Example 3, "ab(cd)" → "abdc"
There's only one pair of brackets here, (cd), which is reversed to give dc. Since this pair isn't nested inside anything else, that single reversal is the only operation needed. Removing the brackets and placing the reversed segment back where it was gives ab + dc = abdc.

Approach to Solve This Problem:

There are two common ways to solve this — a simple stack simulation, and a faster single-pass "direction jump" technique.

Approach 1 — Stack Simulation (Easy to reason about)

1. Walk through the string once. Maintain a stack of indices.
2. On (, push the current length of the result built so far onto the stack.
3. On), pop the last pushed index — that marks where this bracket pair's content starts in the result — and reverse everything in the result from that index to the current end.
4. On any letter, just append it to the result.
5. Once done, the result contains no brackets and is already the final answer.

This is intuitive but can degrade toward **O(N²)** in the worst case, since nested reversals can repeatedly flip large chunks of the result.

Approach 2 — O(N) Direction-Jump Technique (Optimal)

The key insight: instead of physically reversing substrings, we can simulate the effect of nested reversals by changing traversal direction and jumping between matched bracket pairs.

1. **Pair up the brackets:** Do one pass with a stack to record, for every index holding (or), the index of its matching partner.
2. **Traverse with direction control:** Start at index 0 moving rightward (direction = +1).

    - If the current character is a letter, append it to the result and move one step in the current direction.

- If the current character is a bracket, **jump to its matched partner's index** and **flip the direction** (+1 → -1 or -1 → +1), then take one more step in the new direction.

3. Continue until the pointer moves outside the string bounds.
4. The letters collected in the order visited form the final answer directly — no explicit reversal step needed, because flipping direction at each bracket boundary naturally reproduces the effect of reversing every nested segment exactly once.

This turns the problem into a single O(N) traversal with O(N) extra space for the pairing array and the result.

Java Solution:

```java
import java.util.*;

class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();
        System.out.println(reverseParentheses(s));
    }

    static String reverseParentheses(String s) {
        int n = s.length();
        int[] pair = new int[n];
        Deque<Integer> stack = new ArrayDeque<>();

        // Step 1: find matching pairs
        for (int i = 0; i < n; i++) {
            char c = s.charAt(i);
            if (c == '(') {
                stack.push(i);
            } else if (c == ')') {
                int j = stack.pop();
                pair[i] = j;
                pair[j] = i;
            }
        }

        // Step 2: traverse with direction flipping
        StringBuilder result = new StringBuilder();
        int dir = 1;
        for (int cur = 0; cur >= 0 && cur < n; ) {
            char c = s.charAt(cur);
            if (c == '(' || c == ')') {
                cur = pair[cur];
                dir = -dir;
            } else {
                result.append(c);
            }
            cur += dir;
        }

        return result.toString();
    }
}
```

Solution:

1. **Read the input:** the string s is read in a single line.
2. **Match the brackets:** A stack-based pass builds a pair[] array so each bracket knows the index of its partner in O(1).

3. **Traverse with a moving pointer and direction flag:** starting at index 0 heading rightward, letters are appended directly to the result; whenever a bracket is hit, the pointer teleports to its partner and the direction reverses.
4. No explicit substring reversal is ever performed — the direction flips implicitly achieve the same effect as reversing every nested bracket pair from innermost to outermost.
5. **Output:** the collected letters, in visitation order, form the fully reversed and bracket-free string.

Complexity Analysis:

Time Complexity: O(N)

Finding matching pairs takes one linear pass, and the direction-jump traversal visits each character (letter or bracket) a constant number of times, since every bracket is used exactly once to jump and flip direction.

Space Complexity: O(N)

Extra space is used for the pair[] array, the stack (bounded by nesting depth ≤ N), and the result builder — all linear in the input size.